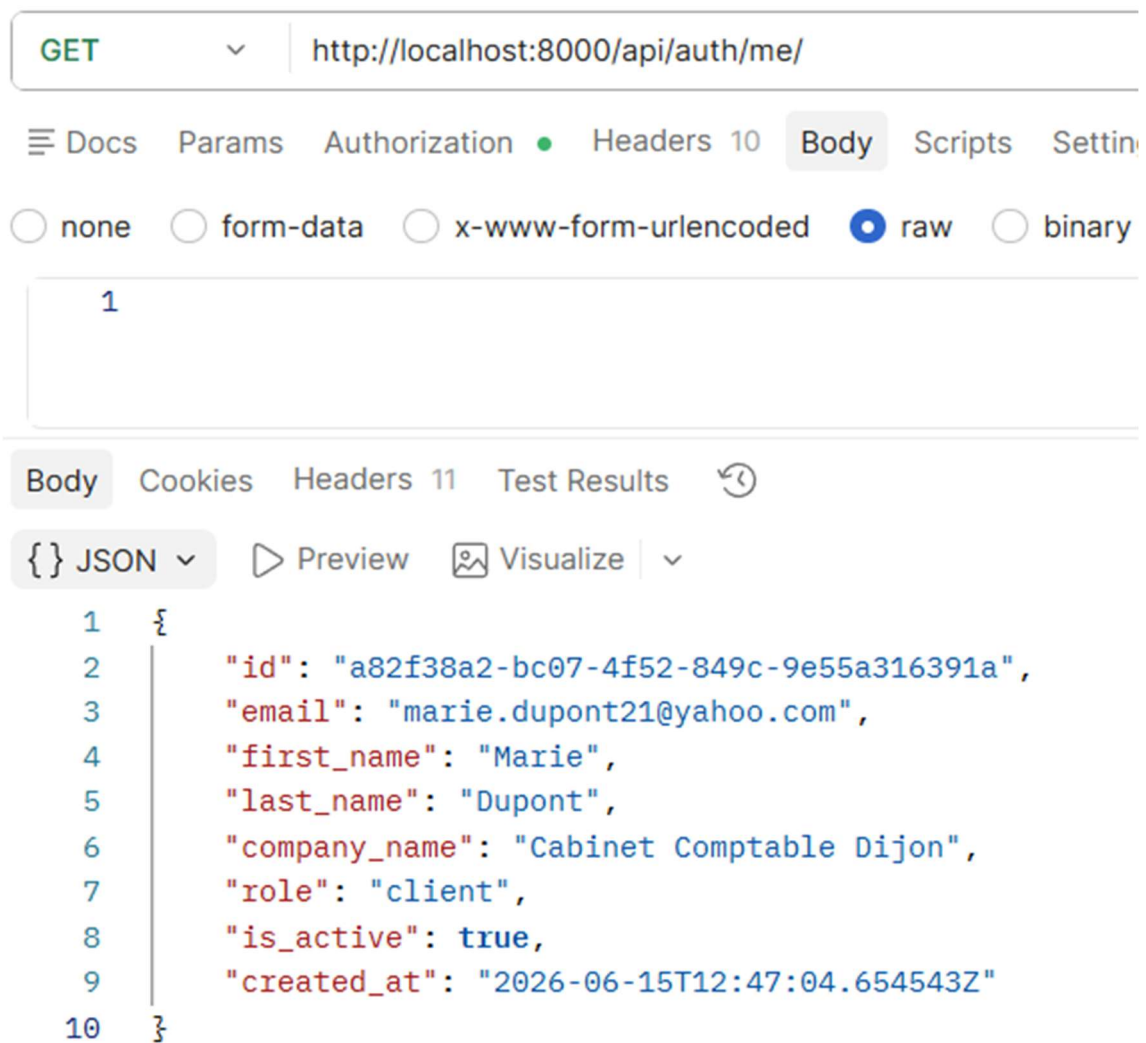


3) View user



Expected result: 200 OK + logged-in user data.

What this test shows:

- The JWT token is successfully validated; the user is identified without sending email/password .
- All profile data is returned: id, email, first_name, last_name, company_name, role , is_active, created_at .
- The password **never appears** in the security best practice response.
- The same UUID as at registration confirms the consistency of the data.

This test therefore validates register → login → me:

The complete JWT authentication chain works end-to-end.

4) Change the user's first name

The screenshot shows a REST client interface with the following details:

- Method:** PATCH
- URL:** http://localhost:8000/api/auth/me/
- Body Type:** raw (selected)
- Body Content:**

```

1  {
2    "first_name": "Marie-Ange"
3  }

```
- Response Body:**

```

1  {
2    "id": "a82f38a2-bc07-4f52-849c-9e55a316391a",
3    "email": "marie.dupont21@yahoo.com",
4    "first_name": "Marie-Ange",
5    "last_name": "Dupont",
6    "company_name": "Cabinet Comptable Dijon",
7    "role": "client",
8    "is_active": true,
9    "created_at": "2026-06-15T12:47:04.654543Z"
10 }

```

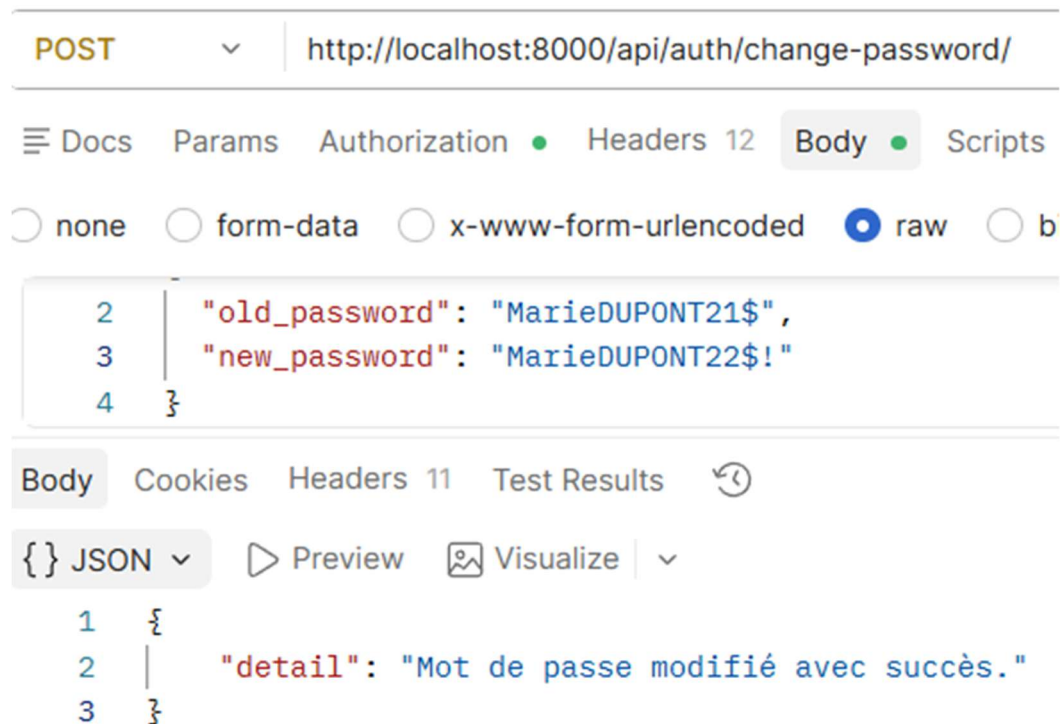
Expected result: 200 OK + profile updated.

What this test shows:

- The partial PATCH works only first_name was sent in the body, and only this field changed ("Marie" → "Marie-Ange").
- All other fields (last_name , company_name , role , etc.) are **unchanged** ; this is exactly the expected behavior of a PATCH.
- The response returns the complete updated profile immediately, without having to repeat a GET /me/.

This test is successful: the partial modification of the profile works correctly and the JWT token correctly identifies the user without passing their ID in the URL.

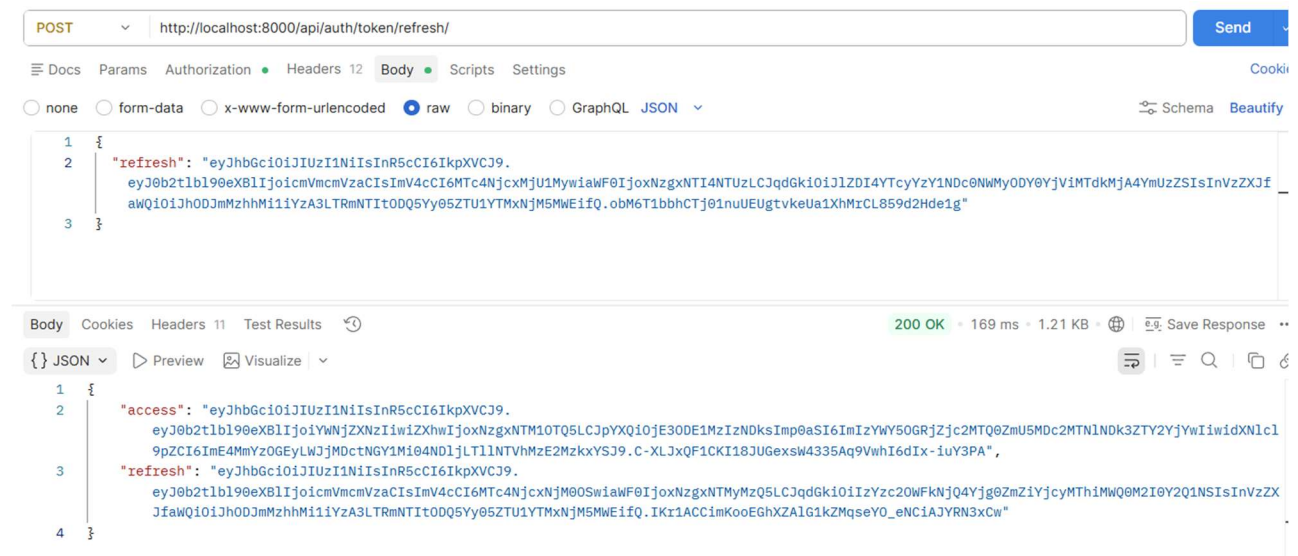
5) Change Password



Expected result: 200 OK + "detail ": "Password changed successfully. " }.

- The old password is verified before allowing the change to be made. Security is correct.
- The new password is accepted and hashed in the database.
- The response is simple and clear: "Password changed successfully." in French, consistent with the rest of the API.
- No sensitive data is returned in the response.

6) Refresh Token



Expected result: 200 OK + new access token.

What this test shows:

- The refresh token has successfully generated a **new access** The previous **token is now replaced**.
- A **new refresh** The **token** is also returned; this is the `ROTATE_REFRESH_TOKENS = True` behavior configured in the Django settings: with each refresh , the previous refresh The token is blacklisted and a new one is issued.
- The old refresh The token sent in the body is no **longer usable** after this request.

This test is valid: automatic token renewal works correctly, useful for maintaining an active session without asking the user for their password again.

Note: Postman now replaces `{{ token }}` and `{{ refresh }}` with the new returned values.

7) Logout (*Blacklist the refresh token*)

POST ▼ http://localhost:8000/api/auth/logout/ Send ▼

Docs Params Authorization Headers 12 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▼ Schema Beautify

```

1  {
2    "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1b190eXB1IjoicmVmcmVzaCI6ImV4cCI6MTc4Njc4NjM0S2IiwiaWF0IjoxNzgzNTMyMzQ5LCJqdGkiOiIiZyZc20WFkNjQ4Yjg0ZmZiYjcyMThiMWQ0M2I0Y2Q1NSIsInVzZXJfaWQiOiJhODJmZzhmMi1iYzA3LTRmNTItODQ5Yy05ZTU1YTMyNjM5MWEifQ.IKr1ACCimKooEGhXZA1G1kZMqseYO_eNCiAJYRN3xCw"
3  }

```

Expected result: 204 No Content

Double-check before sending:

- **Authorization** → Bearer Token with the current token.
- **Body** → the refresh token (the one obtained after the last token / refresh).

The logout should return a **204 No Content**, no body in the response, just an empty status confirming that the refresh token is blacklisted.

PACKS : /api/packs/ (public, no token required)

8) List of packs

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/packs/`. The response is displayed in JSON format, showing a list of two packs. The interface includes tabs for Docs, Params, Authorization, Headers, Body, Scripts, and Settings. The Body tab is selected, and the response is shown in a JSON view. The JSON response is as follows:

```
1  {
2    "count": 4,
3    "next": null,
4    "previous": null,
5    "results": [
6      {
7        "id": 1,
8        "code": "audit",
9        "name": "audit",
10       "description": "help me for audit",
11       "included_services": "Audit & Risk Analysis (EBIOS RM)",
12       "for_whom": "SMEs new to cybersecurity",
13       "perimeter": "Initial assessment, vulnerability report",
14       "duration_days": 5,
15       "price": "1000.00"
16     },
17     {
18       "id": 2,
19       "code": "security",
20       "name": "Security",
21       "description": "help me for security",
22       "included_services": "Audit + Incident Management & Vulnerability Remediation",
23       "for_whom": "SMEs that have experienced an incident",
24       "perimeter": "Audit + remediation of detected vulnerabilities",
25       "duration_days": 10,
26       "price": "2000.00"
27     }
28   ]
29 }
```



```

{
  "id": 3,
  "code": "protection",
  "name": "Protection",
  "description": "help me for protection",
  "included_services": "Audit + System Security & 24/7 SOC Monitoring",
  "for_whom": "SMEs with sensitive data",
  "perimeter": "Audit + Continuous Monitoring & Real-Time Alerts",
  "duration_days": 15,
  "price": "3500.00"
},
{
  "id": 4,
  "code": "premium",
  "name": "Prenium",
  "description": "help me for prenum",
  "included_services": "Audit + Incidents + SOC + Team Awareness",
  "for_whom": "SMEs seeking comprehensive coverage",
  "perimeter": "The most comprehensive all-in-one solution",
  "duration_days": 20,
  "price": "5000.00"
}
]
}

```

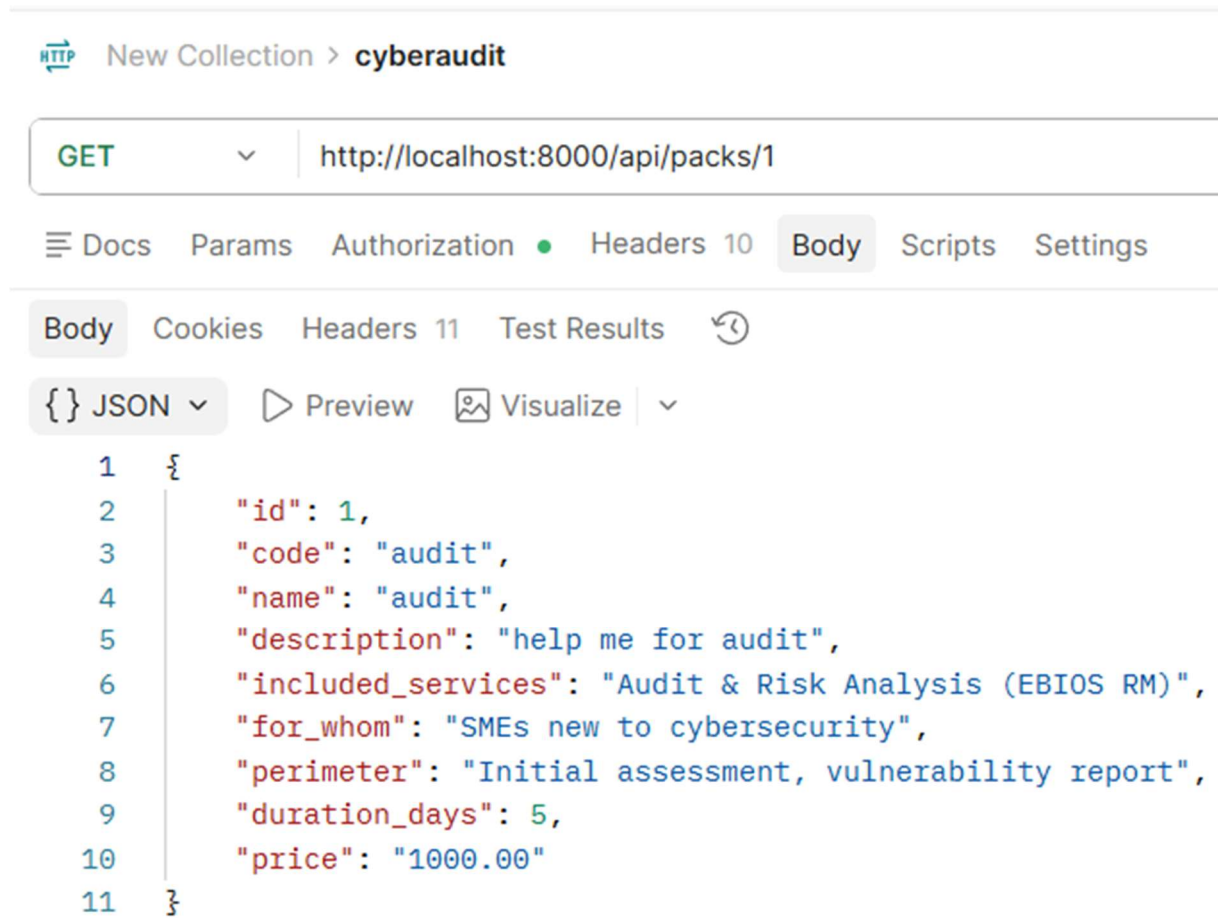
Expected result: 200 OK + list of 4 packs (Audit, Security, Protection, Prenium).

- Pagination is working (next : null , previous : null (everything fits on one page).
- Each pack contains all the expected fields: id, code, name , description, included_services , for_whom , perimeter , duration_days , price .
- This endpoint is **public** , no token required, anyone can see the offers.

The 4 packs created:

id	code	price	duration
1	audit	€1,000	5 days
2	security	€2,000	10 days
3	protection	€3,500	15 days
4	premium	€5,000	20 days

9) Pack detail



HTTP New Collection > cyberaudit

GET http://localhost:8000/api/packs/1

Docs Params Authorization Headers 10 Body Scripts Settings

Body Cookies Headers 11 Test Results

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "code": "audit",
4   "name": "audit",
5   "description": "help me for audit",
6   "included_services": "Audit & Risk Analysis (EBIOS RM)",
7   "for_whom": "SMEs new to cybersecurity",
8   "perimeter": "Initial assessment, vulnerability report",
9   "duration_days": 5,
10  "price": "1000.00"
11 }
```

Expected result: 200 OK + package details (name , included_services, price , duration_days...).

What this test shows:

- The endpoint returns the full details of a single pack by its id.
- **Public** endpoint , no token required.
- All the information is there: included services, target (for_whom), scope, duration, price.

Note: The URL /api/packs/1 without a trailing slash works here because it's a GET request. Django can redirect automatically. But as a best practice, always use the slash:

http://localhost:8000/api/packs/1/

AUDIT REQUESTS: /api/audits/

10) Create an audit request (client only)

<http://localhost:8000/api/audits/>GET <http://localhost:8000/api/packs/>

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/api/audits/`. The request body is a JSON object with the following structure:

```

1  {
2    "pack": 1,
3    "scope_notes": "Nous souhaitons auditer notre infrastructure réseau interne.",
4    "client_info": {
5      "first_name": "Marie",
6      "last_name": "Dupont",
7      "company_name": "Cabinet Comptable Dijon"
8    }
9  }

```

The response is also in JSON format, showing the server's reply:

```

1  {
2    "id": "be672aef-51dd-4706-824f-89214679afb7",
3    "reference": "DOSSIER-2026-0001",
4    "pack": 1,
5    "scope_notes": "Nous souhaitons auditer notre infrastructure réseau interne.",
6    "status": "pending",
7    "submitted_at": "2026-06-15T14:45:48.716041Z"
8  }

```

- The application is correctly registered with a **unique UUID** : be672aef-51dd-4706-824f-89214679afb7.
- A **readable reference** is automatically generated: "DOSSIER-2026-0001" convenient for communicating with the client.
- The initial status is " pending ", awaiting processing by the admin.
- submitted_at is automatically timestamped.
- The JWT token allowed Marie to be automatically identified as a client; there was no need to send her ID in the body.

Note: The client_info fields sent in the body (first_name, last_name, company_name) are not returned in the response; they are likely ignored by the serializer because the client is identified via the token. Only pack, scope_notes, and the automatically generated fields are returned.

11) List of audit requests

New Collection > cyberaudit

GET http://localhost:8000/api/audits/ Send

Docs Params Authorization Headers 10 Body Scripts Settings Coc

raw JSON Schema Beautif

1

Body 200 OK • 55 ms • 1.46 KB • Save Response

{ } JSON Preview Visualize

```

1  {
2    "count": 1,
3    "next": null,
4    "previous": null,
5    "results": [
6      {
7        "id": "be672aef-51dd-4706-824f-89214679afb7",
8        "reference": "DOSSIER-2026-0001",
9        "client": "a82f38a2-bc07-4f52-849c-9e55a316391a",
10       "client_info": {
11         "email": "marie.dupont21@yahoo.com",
12         "first_name": "Marie",
13         "last_name": "Dupont",
14         "company_name": "Cabinet Comptable Dijon"
15       },
16       "pack": {
17         "id": 1,
18         "code": "audit",
19         "name": "audit",
20         "description": "help me for audit",
21         "included_services": "Audit & Risk Analysis (EBIOS RM)",
22         "for_whom": "SMEs new to cybersecurity",
23         "perimeter": "Initial assessment, vulnerability report",
24         "duration_days": 5,
25         "price": "1000.00"
26       },
27       "status": "pending",
28       "scope_notes": "Nous souhaitons auditer notre infrastructure réseau interne.",
29       "internal_notes": "",
30       "assigned_to": null,
31       "submitted_at": "2026-06-15T14:45:48.716041Z",
32       "updated_at": "2026-06-15T14:45:48.716118Z",
33       "completed_at": null
34     }
35   ]
36 }

```

- "count ": 1) Marie **only sees her own requests** (RBAC client), not those of other clients.
- The response is **enriched** compared to the creation: the pack is now a complete object with all its details, and client_info displays the client's data.
- tracking fields are visible : status, internal_notes , assigned_to , submitted_at , updated_at , completed_at

Important fields for the rest of the tests:

Field	Value
id	be 672aef-51dd-4706-824f-89214679afb7
reference	FILE-2026-0001
status	pending
assigned_to	null → not yet assigned to an admin
internal_notes	"" → empty, reserved for admin
completed_at	null → not finished yet

12) Details of a request

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/audits/be672aef-51dd-4706-824f-89214679afb7`. The response is a 200 OK status with a response time of 43 ms and a size of 1.42 KB. The response body is displayed in JSON format, showing details for a specific audit request.

```

1  {
2    "id": "be672aef-51dd-4706-824f-89214679afb7",
3    "reference": "DOSSIER-2026-0001",
4    "client": "a82f38a2-bc07-4f52-849c-9e55a316391a",
5    "client_info": {
6      "email": "marie.dupont21@yahoo.com",
7      "first_name": "Marie",
8      "last_name": "Dupont",
9      "company_name": "Cabinet Comptable Dijon"
10   },
11   "pack": {
12     "id": 1,
13     "code": "audit",
14     "name": "audit",
15     "description": "help me for audit",
16     "included_services": "Audit & Risk Analysis (EBIOS RM)",
17     "for_whom": "SMEs new to cybersecurity",
18     "perimeter": "Initial assessment, vulnerability report",
19     "duration_days": 5,
20     "price": "1000.00"
21   },
22   "status": "pending",
23   "scope_notes": "Nous souhaitons auditer notre infrastructure réseau interne.",
24   "internal_notes": "",
25   "assigned_to": null,
26   "submitted_at": "2026-06-15T14:45:48.716041Z",
27   "updated_at": "2026-06-15T14:45:48.716118Z",
28   "completed_at": null
29 }

```

- The UUID in the URL correctly returns **a single request** with all its complete data.
- Same result as the list but without the pagination directly to the object.
- Marie can access her own request via her client token .

Difference with the previous test (list): here we access **a specific request** by its UUID, useful when the frontend wants to display the details page of a folder.

13) Change status (admin only)

The screenshot shows a REST client interface with the following details:

- Method:** PATCH
- URL:** `http://localhost:8000/api/audits/be672aef-51dd-4706-824f-89214679afb7/`
- Body:** `{ "status": "in_progress" }`
- Response:** 200 OK • 569 ms • 767 B
- Response Body (JSON):**

```

1  {
2    "status": "in_progress",
3    "internal_notes": "",
4    "assigned_to": null,
5    "completed_at": null
6  }

```

What this test shows:

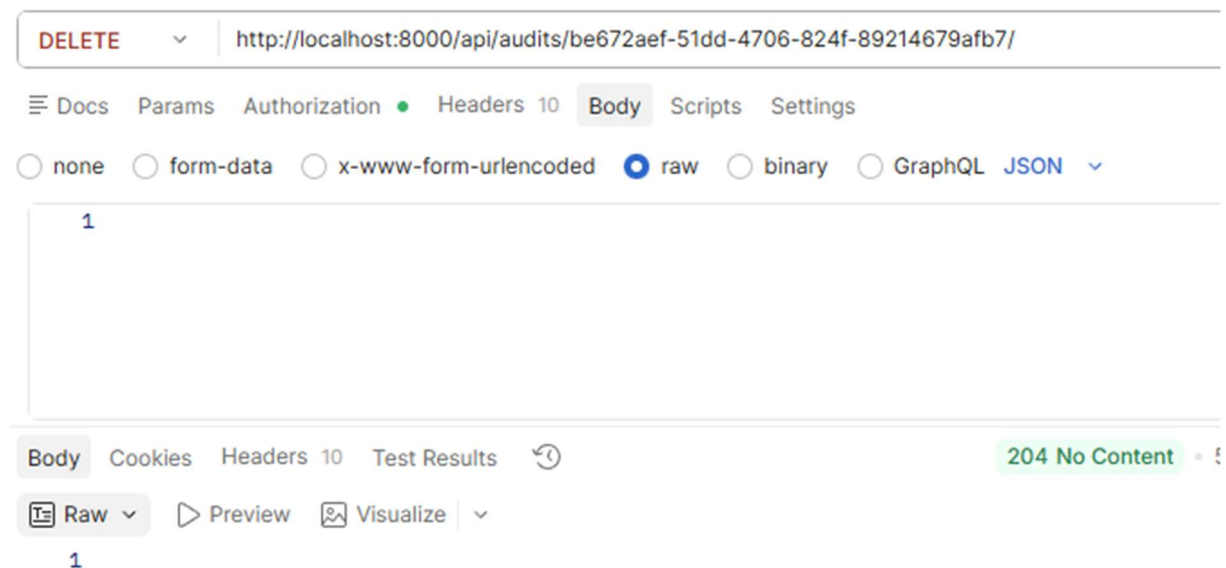
- The status has successfully changed from " pending " → " in_progress ".
- Only fields **that can be modified by the admin** are returned: status , internal_notes , assigned_to , completed_at , this is the AuditRequestAdminUpdateSerializer in action.
- The RBAC works: only an admin token can make this PATCH (a client token would return 403).

What happened in the background: the backend automatically sent an **email notification to Marie** to inform her of the status change; this is the `perform_update` and `create_and_send` logic in `views.py`.

14) Archive a request (admin only)

DELETE http://localhost:8000/api/audits/{id}/

Auth: Bearer {{ token_admin }}



- Status 204 means success **without a body in response**; this is the standard behavior for a DELETE.
- The request is **not deleted** from the database, it is **archived** (soft delete): the status changes to " archived " in the background.
- Only one token **The admin** can perform this action; the RBAC is working correctly.

Why a soft delete ? It's an important architectural choice: audit data is never truly deleted for reasons of traceability and legal compliance. The administrator "archives" a folder rather than permanently deleting it.

15) User notifications

GET http://localhost:8000/api/notifications/me/

Auth: Bearer {{token}}

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8000/api/notifications/me/
- Authorization:** Bearer {{token}}
- Response Status:** 200 OK
- Response Time:** 19 ms
- Response Size:** 1.55 KB
- Response Body (JSON):**

```

1  {
2    "count": 2,
3    "next": null,
4    "previous": null,
5    "results": [
6      {
7        "id": "10b83a48-e9a4-45f4-b293-235f264a8b93",
8        "type": "status_changed",
9        "subject": "Status updated - DOSSIER-2026-0001",
10       "message": "Hello Marie,\n\nYour request DOSSIER-2026-0001 status changed from\n\n\"pending\" to \"in_progress\".\n\nThe CyberAudit & Solutions team.",
11       "status": "sent",
12       "request_reference": "DOSSIER-2026-0001",
13       "sent_at": "2026-06-16T08:33:46.163868Z",
14       "created_at": "2026-06-16T08:33:45.053899Z"
15     },
16     {
17       "id": "862d1b4f-5463-4311-8cef-dc430ea09da7",
18       "type": "request_received",
19       "subject": "Audit request received - DOSSIER-2026-0001",
20       "message": "Hello Marie,\n\nWe have received your audit request\n\n(DOSSIER-2026-0001) for the \"audit\" pack.\n\nYou will receive a\n\nnotification at each status change.\n\nThe CyberAudit & Solutions team.",
21       "status": "sent",
22       "request_reference": "DOSSIER-2026-0001",
23       "sent_at": "2026-06-15T14:45:50.605345Z",
24       "created_at": "2026-06-15T14:45:48.729792Z"
25     }
26   ]
27 }

```

What this test shows:

Marie received 2 automatic notifications related to her file DOSSIER-2026-0001:

Notification 1 "status_changed (today)":

- Sent when the admin changed the status from pending to in_progress .
- Message : "Your request DOSSIER-2026-0001 status changed from 'pending' to 'in_progress'".

Notification 2 “ request_received (yesterday)”:

- Sent automatically when the request is created.
- Message: *"We have received your audit request for the 'audit' pack".*

What this validates:

- The notification system works end-to-end; every action automatically triggers a notification.
- Notifications are linked to the user via the token ; Marie only sees her own.
- The status field : "sent" confirms that the emails have been sent.
- The timestamps sent_at and created_at precisely track each event.

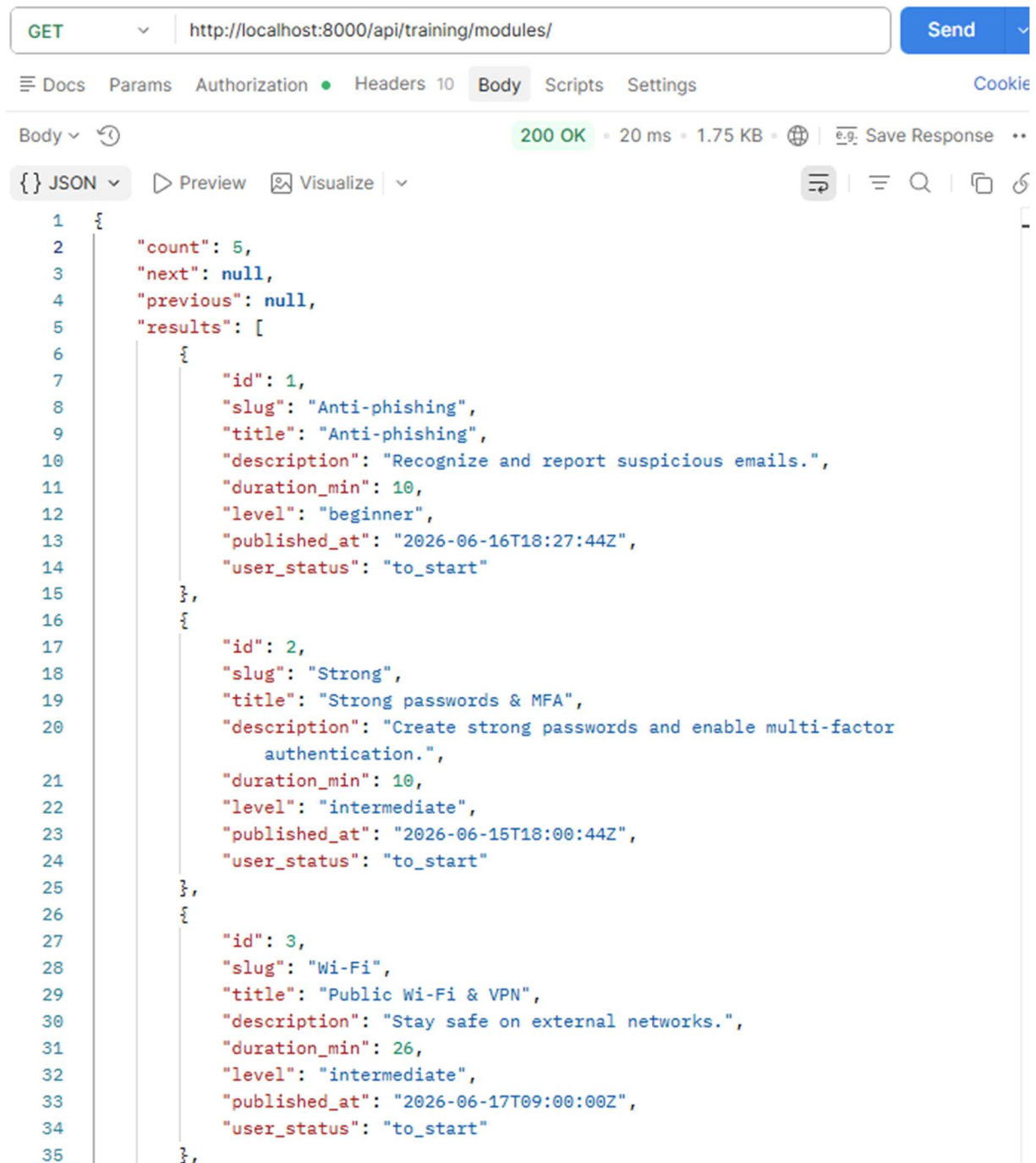
This is one of the most important features of the application: complete traceability of the lifecycle of an audit file.

Training

16) List of modules

GET http://localhost:8000/api/training/modules/

Auth: Bearer {{ token }}



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8000/api/training/modules/
- Authorization:** Bearer {{ token }}
- Status:** 200 OK
- Response Time:** 20 ms
- Response Size:** 1.75 KB
- Body:** JSON

The JSON response body is as follows:

```
{
  "count": 5,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 1,
      "slug": "Anti-phishing",
      "title": "Anti-phishing",
      "description": "Recognize and report suspicious emails.",
      "duration_min": 10,
      "level": "beginner",
      "published_at": "2026-06-16T18:27:44Z",
      "user_status": "to_start"
    },
    {
      "id": 2,
      "slug": "Strong",
      "title": "Strong passwords & MFA",
      "description": "Create strong passwords and enable multi-factor authentication.",
      "duration_min": 10,
      "level": "intermediate",
      "published_at": "2026-06-15T18:00:44Z",
      "user_status": "to_start"
    },
    {
      "id": 3,
      "slug": "Wi-Fi",
      "title": "Public Wi-Fi & VPN",
      "description": "Stay safe on external networks.",
      "duration_min": 26,
      "level": "intermediate",
      "published_at": "2026-06-17T09:00:00Z",
      "user_status": "to_start"
    }
  ]
}
```

```

36 |         {
37 |             "id": 4,
38 |             "slug": "Data",
39 |             "title": "Data backup",
40 |             "description": "Best practices for backing up business data.",
41 |             "duration_min": 10,
42 |             "level": "advanced",
43 |             "published_at": "2026-06-17T12:00:00Z",
44 |             "user_status": "to_start"
45 |         },
46 |         {
47 |             "id": 5,
48 |             "slug": "Incident",
49 |             "title": "Incident response",
50 |             "description": "What to do when something goes wrong.",
51 |             "duration_min": 10,
52 |             "level": "beginner",
53 |             "published_at": "2026-06-20T10:34:24Z",
54 |             "user_status": "to_start"
55 |         }
56 |     ]
57 | }

```

All 5 complete modules are visible:

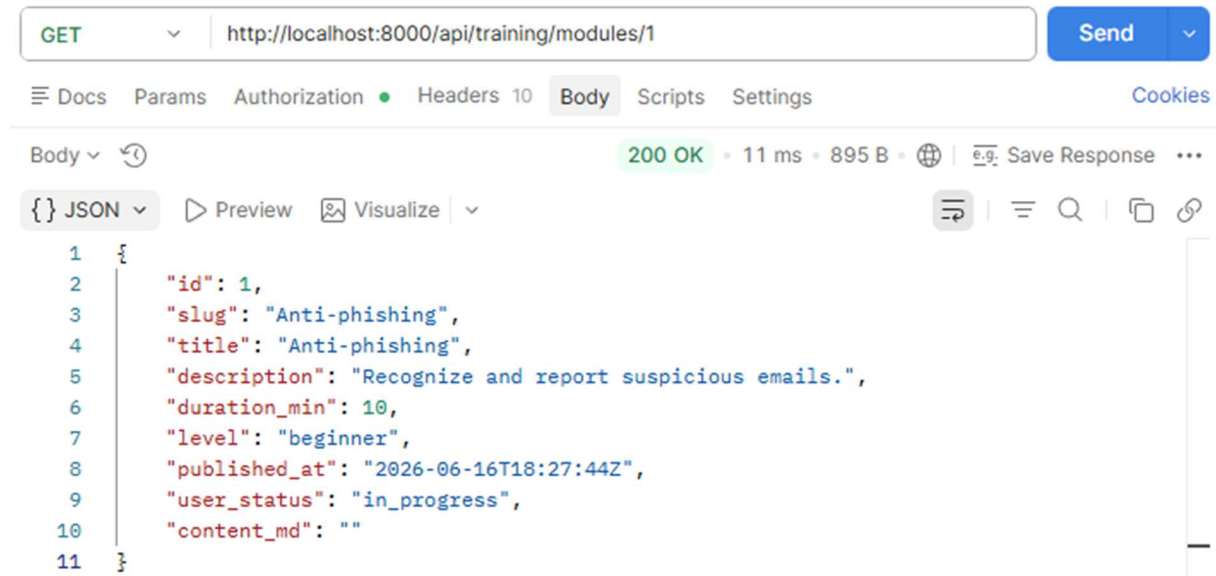
id	title	level	duration
1	Anti-phishing	beginner	10 min
2	Strong passwords & MFA	intermediate	10 min
3	Public Wi-Fi & VPN	intermediate	26 min
4	Data backup	advanced	10 min
5	Incident response	beginner	10 min

Which is perfect: all user_status are "to_start" for Marie, consistent because she has not yet started any module.

17) Detail of a module

GET http://localhost:8000/api/training/modules/1/

Auth: Bearer {{ token }}



- The user _status is now "in_progress", Marie has successfully started the Anti-phishing module.
- The field "content_md": "" appears only in the detail view (not in the list), it is the content of the module in Markdown, empty for now because it has not been entered in the admin.
- Progress is **persisted in the database**; if Marie closes the app and comes back, her in_progress status will still be there.

18) Start a module

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost:8000/api/training/modules/1/start/`
- Buttons:** Send, Docs, Params, Authorization, Headers (11), Body, Scripts, Settings, Cookies
- Response Format:** raw (selected), JSON
- Response Status:** 200 OK, 18 ms, 746 B
- Response Body (JSON):**

```
1 {
2   "status": "in_progress",
3   "started_at": "2026-06-16T12:35:36.479480Z",
4   "completed_at": null
5 }
```

- The progress status changes to "in_progress".
- started_at is automatically timestamped: 2026-06-16T 12:35:36 .
- completed_at : null , the module is not yet finished.

19) Complete a module

The screenshot shows a REST client interface with the following details:

- Collection:** New Collection > cyberaudit
- Method:** POST
- URL:** http://localhost:8000/api/training/modules/1/complete/
- Buttons:** Save, Share, Send
- Tabs:** Docs, Params, Authorization, Headers 11, Body (selected), Scripts, Settings, Cookies
- Body Type:** raw (selected), JSON
- Schema:** Schema, Beautify
- Response:** 200 OK, 19 ms, 770 B, Save Response
- JSON Body:**

```
1 {
2   "status": "completed",
3   "started_at": "2026-06-16T12:35:36.479480Z",
4   "completed_at": "2026-06-16T12:46:11.463690Z"
5 }
```

What this test shows:

- The status has changed to " completed ".
- started_at : 12:35: 36 : start time.
- completed_at : 12:46: 11 : end time.
- Marie took **10 minutes 35 seconds** to complete the Anti-phishing module, consistent with duration_min : 10.

The complete lifecycle of a module is validated: to_start → in_progress → completed .

20) Generate the report as a PDF

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/api/audits/be672aef-51dd-4706-824f-89214679afb7/generate-report/`. The response is a JSON object with the following fields:

```
1 {
2   "id": "7bb50516-8f7e-4dbb-aa8d-cecdf4ef5ed6",
3   "status": "generating",
4   "security_score": 100,
5   "grade": "A",
6   "findings_count": 0
7 }
```

The status bar indicates a **202 Accepted** response.

Status **202** (not 200 or 201) means that the task is **accepted and in progress** ; PDF generation is asynchronous via Celery .

- A report is created with a UUID: 7bb50516-8f7e-4dbb-aa8d-cecdf4ef5ed6.
- " status ": " generating ", the PDF is being created in the background.
- " security_score ": 100 and "grade": "A", automatically calculated score.
- " findings_count ": 0, no vulnerabilities detected (normal, it's a test).

21) Download the PDF report

GET http://localhost:8000/api/audits/be672aef-51dd-4706-824f-89214679afb7/report/

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1

Body Cookies Headers 13 Test Results 200 OK • 18 ms • 13.76 KB

Hex Preview Visualize

CyberAudit & Solutions
Audit Report DOSSIER-2026-0001

Client: Marie Dupont
Company: Cabinet Comptable Dijon
Pack: audit
Date: In progress

Confidential document — Restricted distribution

Executive Summary

A
Security Score : 100/100

Identified Vulnerabilities

No major vulnerability detected.

What this test shows:

- The PDF is returned directly in the response (**13.76 KB**); Postman displays it in Preview.
- The cover page contains all the data for the file:
 - **Client:** Marie Dupont.
 - **Company:** Accounting Firm Dijon.
 - **Package:** audit.
 - **Reference:** DOSSIER-2026-0001.
 - Notice *"Confidential document. Restricted distribution"*.
- The **Executive Summary** poster:
 - Grade **A** , Security Score **100/100**.
 - *"No major vulnerability detected ."*

This test validates the complete chain: request creation → status change → asynchronous PDF generation via Celery → report download.

22) Report data

```

1  {
2    "id": "7bb50516-8f7e-4dbb-aa8d-cecdf4ef5ed6",
3    "audit_reference": "DOSSIER-2026-0001",
4    "summary": "",
5    "verdict": "",
6    "grade": "A",
7    "security_score": 100,
8    "findings": [],
9    "pdf_path": "/home/tommy/portfolio/cyberaudit/backend/media/reports/DOSSIER-2026-0001.pdf",
10   "generated_at": "2026-06-16T13:13:28.556775Z"
11 }

```

200 OK : Report data retrieved!

What this test shows:

- Returns the **JSON metadata** of the report (unlike /report/ which returns the binary PDF).
- audit_reference: DOSSIER-2026-0001, link to the file.
- grade: "A" and security_score: 100, consistent with the generated PDF.
- findings: [] no vulnerabilities recorded.
- pdf_path: exact path of the file on the server → /media/reports/DOSSIER-2026-0001.pdf.
- generated_at: 2026-06-16T13 : 13: 28, generation timestamp.
- The summary and verdict fields are empty; these fields need to be filled in by the admin to enrich the report.

Difference between the two Endpoint reports:

Endpoint	Return
/report/	The binary PDF file (download)
/report/data/	JSON metadata (for frontend display)

All endpoints have now been successfully tested! The CyberAudit & Solutions API is fully functional from end to end.